TRIBHUVAN UNIVERSITY
Institute of Engineering | BCT Program

OPERATING SYSTEM (CT 656)

CHAPTER 6: I/O MANAGEMENT & DISK SCHEDULING

Complete Exam Answer Guide | 4 Hours | Part of 20 Marks (Chapters 6,8,9)

Topic No.	Syllabus Content
6.1	Principles of I/O Hardware
6.2	Principles of I/O Software
6.3	I/O Software Layer
6.4	Disk
6.4.1	Hardware (physical structure, RPM, cylinders, tracks, sectors)
6.4.2	Formatting (low-level, partition, high-level)
6.4.3	Arm Scheduling (FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK)
6.4.4	Error Handling
6.4.5	Stable Storage

Marks Distribution & Exam Appearance

Chapters	Hours	Marks	Frequent Topics
Ch 6, 8, 9	12 combined	20 marks combined	Disk scheduling numericals

Prepared by: Santosh Kushwaha (ACE079BCT)

Question 1: Principles of I/O Hardware

Q: Explain the principles of I/O hardware. What are the different types of I/O devices?

- **Asked in: 2081 Ashwin [2+6] (short note), 2079 Jestha [4], 2078 Chaitra [2+6]**
- **Marks: [2+6] = 8 marks**

1.1 I/O Device Categories

Category	Type	Examples
Block Devices	Store data in fixed-size blocks; each block can be read/written independently	Disk drives, USB drives, CD-ROM
Character Devices	Deliver or accept a stream of characters; not addressable	Keyboard, mouse, printer, serial ports
Network Devices	Hybrid — neither purely block nor character	Network Interface Card (NIC), WiFi

1.2 I/O Hardware Components

- Port: Connection point for device (USB port, serial port)
- Bus: Shared communication link (PCIe, USB bus, SATA)
- Controller: Electronic component managing device operations (disk controller, USB controller)
- Device Registers: Command, Status, Data-in, Data-out registers in controller

CPU communicates with I/O controllers via:

- Port-Mapped I/O: Special IN/OUT instructions; device registers have separate address space
- Memory-Mapped I/O: Device registers mapped to regular memory addresses; standard load/store instructions used

Question 2: I/O Software — Principles and Layers

Q: What are the principles of I/O software? Explain the layers of I/O software with their responsibilities.

- **Asked in: 2081 Ashwin [short note], 2079 Jestha [2+6], 2078 Chaitra [2+6], 2080 Chaitra [2+6]**
- **Marks: [2+6] = 8 marks**

2.1 Goals of I/O Software

- Device Independence: Programs work with any device without modification
- Uniform Naming: Devices accessed by name (e.g., /dev/sda), not by hardware address
- Error Handling: Errors handled as close to hardware as possible (in driver)
- Synchronous vs Asynchronous I/O: Blocking (wait) or non-blocking (continue immediately)
- Buffering: Temporary storage to handle speed mismatch between producer/consumer
- Sharable vs Dedicated Devices: Printers dedicated; disks shared

2.2 I/O Software Layers

Layer	Responsibility	Example
User-Level I/O Software	Library calls, standard I/O formatting, spooling	printf(), fread(), write()
Device-Independent OS Software	Buffering, error reporting, device naming, block size uniformity	Buffer cache, /dev interface
Device Drivers	Device-specific code; talks to controller registers	disk_driver.c, usb_driver.ko
Interrupt Handlers	Notifies driver when I/O completes; wakes waiting process	IRQ handler, DMA completion
Hardware	Actual device; raises interrupt when done	Disk, keyboard, NIC

Question 3: Types of I/O — Programmed, Interrupt, DMA

Q: What is DMA? Explain how I/O takes place through DMA. Also explain Programmed I/O.

- **Asked in: 2082 Kartik [3+6], 2079 Chaitra [2+8], 2080 Chaitra [2+8], 2079 Ashwin [3+4]**
- **Marks: [3+6] = 9 marks**

3.1 Programmed I/O (Polling / Busy-Wait)

In programmed I/O, the CPU directly controls I/O operations and waits (busy-waits) until the device is ready.

How it works:

1. CPU writes I/O command to device register
2. CPU continuously polls (reads) status register until DONE bit is set
3. CPU reads data from data register
4. CPU repeats until all data transferred

Disadvantage: CPU wastes time polling — cannot do other work. Inefficient for slow devices like printers.

3.2 Interrupt-Driven I/O

In interrupt-driven I/O, the CPU initiates I/O and continues other work. Device interrupts CPU when done.

How it works:

1. CPU writes I/O command to controller; saves context
2. CPU schedules another process — does useful work
3. Device raises interrupt when I/O completes
4. CPU saves state, runs interrupt handler
5. Handler transfers data; original process resumes

Advantage: CPU productive while I/O runs. Disadvantage: Interrupt overhead for high-speed devices.

3.3 DMA — Direct Memory Access

DMA (Direct Memory Access) is a hardware mechanism that allows I/O devices to transfer data directly to/from memory without CPU involvement.

DMA Operation Steps:

1. CPU programs DMA controller: source address, destination address, byte count, direction
2. CPU initiates DMA transfer and continues executing other processes
3. DMA controller directly transfers data between device and memory (via bus)
4. DMA steals memory bus cycles from CPU (cycle stealing) — CPU slightly slower but still runs
5. DMA raises interrupt when transfer complete
6. CPU processes interrupt — data already in memory

Method	CPU Usage	Speed	Suitable For
Programmed I/O	100% (busy-waiting)	Slow	Very slow devices
Interrupt-Driven	Low (only during interrupt)	Medium	Moderate speed devices
DMA	Minimal (cycle stealing)	Very Fast	High-speed: disk, network, GPU

Question 4: Disk Structure & Arm Scheduling Algorithms

Q: Explain about the direct memory access. Suppose that a disk drive has 100 cylinders, numbered 0 to 99. The drive is currently serving a request at cylinder 35 and previous request was at cylinder 20. The queue of pending request in FIFO order is 86, 70, 13, 74, 48, 9, 22, 50, 30. Produce the schedules to satisfy all the pending requests for each of the following disk scheduling algorithm: (a) C-LOOK (b) SCAN (c) C-SCAN

Asked in: 2082 Kartik [3+6]

4.1 Disk Structure

- Platter: Circular magnetic disk (a drive may have multiple platters)
- Track: Concentric circle on a platter surface
- Cylinder: Set of all tracks at same position across all platters
- Sector: Smallest unit of data storage (typically 512 bytes or 4096 bytes)
- Seek Time: Time to move arm to correct cylinder (dominant cost)
- Rotational Latency: Time for sector to rotate under head
- Transfer Time: Time to read/write data

4.2 Solved Disk Scheduling (100 cylinders, head at 35, prev at 20)

Head at: 35 | Previous request: 20 | Direction: MOVING UP (since 20 → 35 is increasing)

Request Queue (FIFO): 86, 70, 13, 74, 48, 9, 22, 50, 30

Sorted request positions: 9, 13, 22, 30, 48, 50, 70, 74, 86

(a) FCFS — Serve in arrival order:

```
Order: 35 → 86 → 70 → 13 → 74 → 48 → 9 → 22 → 50 → 30
|35-86|=51, |86-70|=16, |70-13|=57, |13-74|=61, |74-48|=26, |48-9|=39, |9-22|=13, |22-50|=28, |50-30|=20
```

FCFS Total Head Movement = 51+16+57+61+26+39+13+28+20 = 311 cylinders

(b) SCAN — Move up to end (99), then reverse down:

```
Order: 35 → 48 → 50 → 70 → 74 → 86 → 99 → 30 → 22 → 13 → 9
35→48(13), 48→50(2), 50→70(20), 70→74(4), 74→86(12), 86→99(13), 99→30(69), 30→22(8), 22→13(9), 13→9(4)
```

SCAN Total = 13+2+20+4+12+13+69+8+9+4 = 154 cylinders

(c) C-SCAN — Move up to end (99), jump to 0, continue up:

```
Order: 35 → 48 → 50 → 70 → 74 → 86 → 99 → [jump to 0] → 9 → 13 → 22 → 30
35→86(51 going up), 86→99(13), jump 99→0(99 cylinders), 0→30(30)
```

C-SCAN Total = (99-35) + (99-0) + (30-0) = 64+99+30 = 193 cylinders

(d) C-LOOK — Move up to highest request (86), jump to lowest (9), continue up:

```
Order: 35 → 48 → 50 → 70 → 74 → 86 → [jump to 9] → 9 → 13 → 22 → 30
35→86(51), jump 86→9(77), 9→30(21)
```

C-LOOK Total = 51+77+21 = 149 cylinders

Algorithm	Total Head Movement	Key Characteristic
FCFS	**311 cylinders**	Simple; no optimization; can zigzag badly
SCAN (Elevator)	**154 cylinders**	Sweeps one direction, then reverses
C-SCAN	**193 cylinders**	Uniform wait; always restarts from beginning
C-LOOK	**149 cylinders**	Efficient variant; only goes to actual requests

Question 5: Comprehensive Disk Scheduling (2081 Ashwin Exam)

Q: Suppose a disk drive with 150 cylinders numbered from 0-149. The drive is currently serving a request at cylinder 35 and the previous request is at disk 55. The queue of pending request is 98, 103, 38, 122, 10, 128, 65, 75. Starting from the current head position, calculate the total head moment (in cylinder) that the disk arm moves to satisfy all the pending request for SSTF, SCAN and LOOK Disk Arm Scheduling Algorithm.

- **Asked in: 2081 Ashwin [6], 2079 Jestha [6], 2079 Ashwin [10], 2081 Chaitra [3+6]**
- **Marks: [6] marks**

Head at: 35 | Previous request: 55 | Direction: MOVING DOWN (since 55 → 35 is decreasing)

Request Queue: 98, 103, 38, 122, 10, 128, 65, 75

Sorted: 10, 38, 65, 75, 98, 103, 122, 128

FCFS — Serve in arrival order:

```
35 → 98 → 103 → 38 → 122 → 10 → 128 → 65 → 75
|35-98|=63, |98-103|=5, |103-38|=65, |38-122|=84, |122-10|=112, |10-128|=118, |128-65|=63, |65-75|=10
```

FCFS Total = 63+5+65+84+112+118+63+10 = 520 cylinders

SSTF (Shortest Seek Time First) — Always serve nearest cylinder:

```
35 → 38 → 65 → 75 → 98 → 103 → 122 → 128 → 10
|35-38|=3, |38-65|=27, |65-75|=10, |75-98|=23, |98-103|=5, |103-122|=19, |122-128|=6, |128-10|=118
```

SSTF Total = 3+27+10+23+5+19+6+118 = 211 cylinders

SCAN — Moving DOWN from 35, go to 0, then reverse UP:

```
35 → 10 → 0 → 38 → 65 → 75 → 98 → 103 → 122 → 128
|35-10|=25, |10-0|=10, |0-38|=38, |38-65|=27, |65-75|=10, |75-98|=23, |98-103|=5, |103-122|=19, |122-128|=6
```

SCAN Total = 25+10+38+27+10+23+5+19+6 = 163 cylinders

LOOK — Moving DOWN from 35, go to lowest request (10), then reverse UP:

```
35 → 10 → 38 → 65 → 75 → 98 → 103 → 122 → 128
|35-10|=25, |10-38|=28, |38-65|=27, |65-75|=10, |75-98|=23, |98-103|=5, |103-122|=19, |122-128|=6
```

LOOK Total = 25+28+27+10+23+5+19+6 = 143 cylinders

Algorithm	Total Movement	Sequence
FCFS	**520 cylinders**	35→98→103→38→122→10→128→65→75
SSTF	**211 cylinders**	35→38→65→75→98→103→122→128→10
SCAN	**163 cylinders**	35→10→0→38→65→75→98→103→122→128
LOOK	**143 cylinders**	35→10→38→65→75→98→103→122→128

LOOK is most efficient here. FCFS is worst. SCAN and LOOK provide more uniform service than SSTF (no starvation).

Question 6: Large-Scale Disk Scheduling (2000 Cylinders)

Q: Suppose a disk drive has 2000 cylinders, numbered 0 to 1999. The drive is currently serving a request at cylinder 134 and the previous request was at cylinder 124. The queue of pending requests in FIFO order is 86, 1470, 913, 1774, 948, 1509, 1022, 1750, 130. Starting from the current head position what is the total distance that disk arm moves to satisfy all the pending requests for each of the following disk scheduling algorithms? FCFS, SSTF, SCAN, C-SCAN, LOOK

Asked in: 2080 Ashwin [10]

Head at: 134 | Previous: 124 | Direction: MOVING UP (124 → 134 is increasing)

Queue: 86, 1470, 913, 1774, 948, 1509, 1022, 1750, 130

Sorted: 86, 130, 913, 948, 1022, 1470, 1509, 1750, 1774

FCFS:

```
134→86→1470→913→1774→948→1509→1022→1750→130
|134-86|=48, |86-1470|=1384, |1470-913|=557, |913-1774|=861, |1774-948|=826, |948-1509|=561, |1509-1022|=487, |1022-1750|=728, |1750-130|=1620
```

FCFS Total = 48+1384+557+861+826+561+487+728+1620 = 7072 cylinders

SSTF:

```
134→130→86→913→948→1022→1470→1509→1750→1774
|134-130|=4, |130-86|=44, |86-913|=827, |913-948|=35, |948-1022|=74, |1022-1470|=448, |1470-1509|=39, |1509-1750|=241, |1750-1774|=24
```

SSTF Total = 4+44+827+35+74+448+39+241+24 = 1736 cylinders

SCAN (moving UP, go to 1999, then reverse):

```
134→130→86→[reverse]→913→948→1022→1470→1509→1750→1774
```

Wait — direction is UP. Serve in UP direction first:

```
134→913→948→1022→1470→1509→1750→1774→1999→[reverse]→130→86
(1999-134) + (1999-86) = 1865 + 1913 = no...
```

Corrected SCAN (moving UP from 134):

```
Up: 134→913(779)→948(35)→1022(74)→1470(448)→1509(39)→1750(241)→1774(24)→1999(225)
Reverse down: 1999→130(1869)→86(44)
```

SCAN Total = (1999-134) + (1999-86) = 1865 + 1913 = actually:

SCAN Total = 1865 + (1999-86) = 1865+... let me recalculate:

```
Up from 134 to 1999: 1999-134 = 1865 cylinders
Down from 1999 to 86: 1999-86 = 1913 cylinders
```

SCAN Total = 1865 + 1913 = 3778 cylinders

LOOK (moving UP, only to highest request 1774, then reverse):

```
Up: 134→913→948→1022→1470→1509→1750→1774
1774-134 = 1640 cylinders (going up)
Down: 1774→130→86
1774-86 = 1688 cylinders (going down)
```

LOOK Total = 1640 + 1688 = 3328 cylinders

C-SCAN (moving UP, go to 1999, jump to 0, continue UP):

```
134→...→1774→1999 [jump to 0] → 86→130
(1999-134) + (1999-0) + (130-0) = 1865+1999+130 = 3994 cylinders
```

C-SCAN Total = 3994 cylinders

Algorithm	Total Movement	Sequence
FCFS	**7072 cylinders**	Arrival order — very poor
SSTF	**1736 cylinders**	Always nearest — best here but risk of starvation
SCAN	**3778 cylinders**	Up to 1999, reverse down
C-SCAN	**3994 cylinders**	Up to 1999, jump to 0, up to 130
LOOK	**3328 cylinders**	Up to 1774, reverse down to 86

Question 7: Disk Scheduling Algorithm Comparison

Q: Write short notes on: (a) Principles of I/O Software (b) DMA (c) Stable Storage (d) SSTF vs SCAN disk scheduling

- **Asked in: 2081 Ashwin [3×3], 2079 Jestha [3×3], 2078 Chaitra [3×3]**
- **Marks: [3×3] = 9 marks**

7.1 Comprehensive Algorithm Comparison

Algorithm	Strategy	Performance	Issue
FCFS	First-come, first-served	Poor for spread requests	Wild arm movement; no optimization
SSTF	Nearest cylinder first	Best average performance	Starvation of far cylinders
SCAN	Elevator: one direction then reverse	Good, uniform service	Bias: middle cylinders get more visits
C-SCAN	One direction only, then jump	Uniform wait time	Overhead of jump (no service on return)
LOOK	Like SCAN but to last request	Better than SCAN	Slightly complex to implement
C-LOOK	Like C-SCAN but to last request	Most efficient in practice	Minor implementation complexity

7.2 Stable Storage

Stable storage is a storage system designed to survive any single failure (disk crash, power failure). Implemented using RAID or dual-disk system:

- Write: Write to both disks (mirrored). If first write fails, retry until success.
- Read: Read from first disk. If error, read from second.
- Crash recovery: Compare both disks; repair inconsistency using good copy.

Goal: Either complete the write to both disks, or restore to original state — atomicity guaranteed.

7.3 Error Handling on Disk

- Bad Blocks: Sectors that are permanently damaged — mapped out using spare sectors
- ECC (Error-Correcting Code): Extra bits per sector to detect and correct errors
- Sector Slipping: Remapping bad sector to next available good sector
- Write Verification: After write, read-back to verify data was written correctly